$$\Rightarrow \text{using chain-rule: } \frac{\partial L}{\partial W_2[n,k]} = \sum_i \frac{\partial L}{\partial h_2[i,k]} \cdot \frac{\partial h_2[i,k]}{\partial W_2[n,k]}$$

$\Rightarrow$ substitute "gradient formulas" above (previous)

$$= \sum_{i=0}^{bs} h_2 \cdot grad[i,k] * h_1[i,n]$$

$\Rightarrow$ finally

$$W_2 \cdot grad[n,k] = \sum_i h_1[i,n] \cdot h_2 grad[i,k]$$

same as

$$\Rightarrow \quad W_2 \cdot grad[i,j] = sum\_i\, h_1[i,j] \cdot h_2 grad[i,k]$$

- For equation #2: $h_1 \cdot grad$ = "how much did this intermediate feature contribute to the final ~~error~~ error in all outputs for sample i?"

To deduce equation #2:

what we already had: $\frac{\partial h_2[i,k]}{\partial h_1[i,j]} = W_2[j,k]$ — substitute

applying chain-rule: $\frac{\partial L}{\partial h_1[i,j]} = \sum_k \frac{\partial L}{\partial h_2[i,k]} \cdot \frac{\partial h_2[i,k]}{\partial h_1[i,j]}$

$$= \sum_k h_2 \cdot grad[i,k] \cdot W_2[j,k]$$

- finally: $\Rightarrow$

$$h_1 \cdot grad[i,j] = \sum_k W_2[j,k] \cdot h_2 \cdot grad[i,k]$$

same as

$$h_1 \cdot grad[i,j] = sum\_k\, W_2[j,k] * h_2 \cdot grad[i,k]$$

Models: build Models with Pytorch.

nn.parameters()

x = nn.parameter(torch.randn(input_dim))

output = x @ w

assert oueput.size() == torch.size([hidden_dim])

!! rescale by 1/sqrt(num_inputs)

w = nn.Parameter(torch.randn(input_dim, hidden_dim)/np.sqrt(input_dim))

oueput = x @ w

---

custom_model()

data_loading()

optimizer(): ~~Adam~~ ✓, W-Adam, ~~Adagrad~~ ✓ AdaGrad, SGD ✓

Momemet,

Memory: → params: (D*p* num_layers) + D

→ Activations: B*D* num_layers

data => ↓ gradienes = num of params

↓ optimizer = num of params

total memory = 4* (num params + num Activations + num gradienes + num_optimizer)

---

compute: flops = 6 * B * num_params